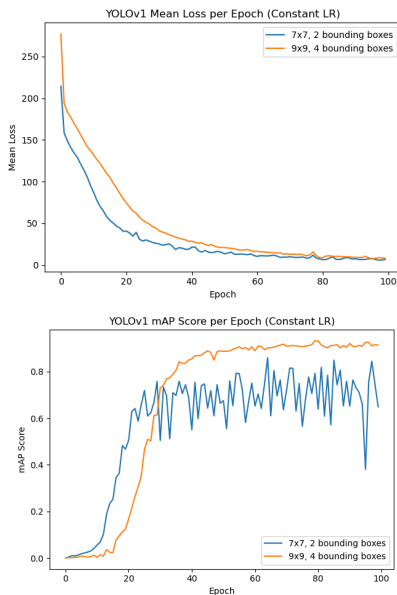


CS190I Programming Assignment 1

Simon Yan

The foundational YOLOv1 model algorithm and loss function are based off of Aladdin Persson's YOLOv1 from Scratch Youtube video. The model is a simplified version of the original YOLOv1 model, splitting images into 7x7 cells with 2 bounding boxes in each cell and a fully connected layer of 496 instead of 4096 to reduce training time. The model is also trained on the PASCAL VOC dataset, with ~16,000 training images and ~5,000 testing images.

My first modification of the YOLOv1 implementation was to change the split size and anchor boxes. One of the main limitations of the YOLO algorithm is that each bounding box can only predict a singular object. Thus, if an image has a lot of small objects, only one in each cell can be detected. I opted to increase the grid size from 7x7 to 9x9, and add 2 additional bounding boxes to 4 total, allowing the algorithm to test more



shaped anchors, like taller boxes, skinner boxes, and small squares. The main trade-off of larger values is the training time. However, both of the models trained at the same time, with about 6 minutes 15 seconds per epoch.

As you can see in the two graphs to the left, the modified YOLOv1 implementation with 9x9 cells and 4 bounding boxes produces a much

higher mAP score, while the mean loss is slightly lower than the original implementation of YOLOv1.

mAP score (mean average precision) is the main evaluation metric for object detection models. It uses precision and recall to calculate both the accuracy of class predictions as well as the completeness, measuring how well the model found all the objects in the images. Mean loss is the average batch loss of class predictions while training. We will use mAP score as the main indicator of how good a model is, as it is used for validation and testing.

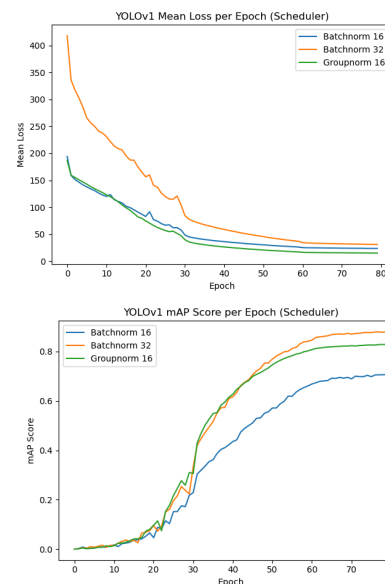
The 7x7 cells and 2 bounding boxes model reaches convergence at about 30 epochs and then fluctuates up and down, meaning the learning rate is too high and the model is at

risk of overfitting. In my next modifications, I look at adding a dynamic learning rate.

The original YOLOv1 paper uses a scheduler to dynamically change the learning rate between epochs because the model has more to learn in the first 30 epochs and needs to reduce learning rate to make smaller changes and avoid overshooting. My next modifications all use a learning rate scheduler, and I tested different model architectures and hyperparameters.

The foundation YOLOv1 model uses BatchNorm2D with batch size of 16. However, BatchNorm2D is more efficient with larger batch sizes. I also tested GroupNorm, which is more efficient with smaller batch sizes, keeping the same batch size of 16. The results show that BatchNorm2D with a larger batch size is not only more precise with a higher mAP score, but also takes less training time, a 15% decrease in

training time from 6 minutes 15 seconds to 5 minutes 20 seconds per epoch. It was also better than GroupNorm, showing that it is a more efficient normalization strategy that learns better based on the batch size compared to GroupNorm, which is not dependent on batch size and performs the same for all batch sizes.



Conclusion

My modifications of the foundational YOLOv1 model based off of Aladdin Persson's design improved the mAP score by more than 10%, with much more stable training, and a shorter training time. However, due to hardware constraints, I was not able to train a final model combining all of my modifications. My modifications align with later YOLO models, which add additional bounding boxes and more complex cell splitting, such as YOLOv3, which uses multi-scale cell structures to add additional complexity understanding to the model.

References

<https://arxiv.org/pdf/1506.02640>
https://www.youtube.com/watch?v=n9_XyCGr-MI
https://github.com/simony05/cs190i_prog1